Type pg_stat_statements inside the single quotes using your editor. Once you've done this, save the changes to the file and exit. After your changes the line should look like this:

sh/shared_preload_libraries.sh
```
shared_preload_libraries = 'pg_stat_statements'
```

This change does require a restart. In your production database server, a restart needs to be carefully planned. Other customizations require only a "reload" and not a restart which is much better because it doesn't interrupt any running database queries or activity.

Since you're working locally and are the sole user of the database, restarting is no big deal. If you're using PostgresApp on Mac OS, you can restart PostgreSQL using the GUI. Click "Stop" and "Start" from the GUI app.

To restart PostgreSQL from your terminal, use the pg_ctl program and restart argument. This command expects PGDATA to be set either as an environment variable, or as a command argument. The value should be the path to your data directory for your installation.

To get the path to your data directory, run the following command from psql:

```
SHOW data_directory;
```

Type \q to exit psql and return to your terminal.

You can now run pg_ctl restart and use --pgdata with a directory path like "/Users/andy/Library/Application Support/Postgres/var-15".

The full command to restart PostgreSQL in your terminal is:

```
pg_ctl restart -D "/Users/andy/Library/Application Support/Postgres/var-15"
```

Once PostgreSQL is restarted, run psql again and run the following query to confirm PGSS is listed:

sql/select_pg_extension.sql
```
SELECT * FROM pg_extension;
```

You should see pg_stat_statements in the extname column.

Great. You've now tried some basics on configuring an extension. Next you'll start to explore what's currently happening in your database while it's running.

Getting Started With Query Observability

By looking at the schema, tables, relationships and structure, you can get an idea of what's being written into the database.